bcc09 for OS-9 Programmers

C for OS-9 and NitrOS-9, for those who know the system

bcc09 project

November 2026

Contents

1 Why bcc09? **1**

2 Your first module **2**

3 How the code runs **3**

4 The command line and the exit **4**

5 Lines, the terminal and files **5**

6 Memory **7**

7 System calls of your own **7**

8 What's there, and what isn't **8**

9 Testing **9**

Appendix: from OS-9 assembly to C **9**

1 Why bcc09?

If you have used OS-9 on a Color Computer, or NitrOS-9 today, you know its modules, its paths and its shell, and perhaps its own C compiler, run on the machine itself. *bcc09* is a C compiler for the 6809 that runs on a PC instead: it compiles a program there, in seconds, into an OS-9 program module that you copy to a disk and run from the shell like any other command.

- **The modules are OS-9's own kind:** position-independent and reentrant, so one copy in memory serves every process that runs it.

- **The C is C89,** with prototypes, `void`, structure assignment and the rest, and a C library with `stdio`, `string.h`, `stdlib.h`, `malloc`, and `math.h`, built for OS-9.
- **The system is there:** files and directories through RBF, the terminal through SCF, the heap through `F$Mem`, and any system call you need through a few lines of assembly.

This guide is for programmers who know OS-9 and some C. It shows how bcc09 builds a module, how a C program meets OS-9, and what to watch for. The bcc09 user's manual has the rest of the compiler and its library.

1.1 What you need

- **bcc09**, on a PC running Linux: on Fedora, the RPM from bcc09's releases on GitHub; otherwise, built from its source.
- **toolshed's `os9` command**, to copy modules to an OS-9 disk image (and `os9 ident` to look at them).
- **OS-9 to run them on:** NitrOS-9 on an emulated CoCo 3 (XRoar or MAME), or a real machine, with the module copied over on a disk. bcc09's tests run their modules on NitrOS-9 Level 2. The modules are the same for Level 1 and Microware's OS-9 Level 2, which the tests don't run.

2 Your first module

```c
#include <stdio.h>

int main()
{
    printf("HELLO FROM C\r");
    return 0;
}
```

Compile it, make it a module, and put it on a disk image:

```
bcc09 -Mo -O -o hello.out hello.c
objcopy09 -O os9 -n hello hello.out hello
os9 copy -r hello nitros9.dsk,CMDS/hello
os9 attr -e -pe nitros9.dsk,CMDS/hello
```

- `-Mo` is the OS-9 target: the code generation, the start-up code, the C library built for OS-9, and the layout of a module. `-O` runs the peephole optimizer.
- `objcopy09 -O os9` makes the module from the linker's output: `-n` is its name. `-d` sets the stack's size (1K by default).
- `os9 copy` puts it on the disk, and `os9 attr` makes it executable, for its owner and the others.

Boot the disk, and `hello` runs from the shell like any command in `CMDS`.

2.1 What's in the module

```
$ os9 ident hello               (in part)
Header for : hello
Module size: $175D  #5981
Module CRC : $080F57 (Good)
Hdr parity : $2C
Exec. off  : $000D  #13
Data size  : $06D8  #1752
Ty/La At/Rv: $11 $80
Prog mod, 6809 Obj, re-ent, R/O
```

The header, the code, an image of the initialized data, the name and the CRC. The data size is the permanent storage OS-9 gives each process running the module: the program's data and uninitialized data (BSS), then the stack. The start-up code copies the data's initial values from the module to the data area, clears the BSS, and calls `main`.

Most of those 5981 bytes are `printf`, about 4K. The same program with `write(1, "HELLO FROM C\r", 13)` instead is a module of 1201 bytes.

3 How the code runs

3.1 Position-independent, data through U

OS-9 loads a module wherever it finds room, and runs one copy of it for every process, each with its own data area, which `F$Fork` gives the process with U pointing at it. bcc09's code for OS-9 is made for that:

- **Calls and constants are relative to the PC:** `LBSR` for calls, `LEAX label,PC` for strings, so the code runs wherever it is loaded.
- **Variables are offsets from U:** the data area starts at offset 0, and a global `count` is `count,U`. U is never used for anything else.
- **DP is the data area's page,** which `F$Fork` sets; the first 256 bytes of the data area can be reached in one byte as `<name`, for variables declared `__direct` (section 3.3).

```
int count;

void tick()
{
    ++count;
}
```

becomes, with `-O`,

```
_tick:  LDD     _count,U
        ADDD    #1
        STD     _count,U
        RTS
```

3.2 Static pointers

A static or global variable that holds an address can't have its value in the module's image of the data, since the address isn't known until the module is loaded. bcc09 makes code that sets it, in a function the start-up code calls before `main`:

```
char *greeting = "HELLO\r";
```

gives `greeting` 0 in the data's image, and then

```
        LEAX    .1,PC           ; the string, wherever the module is
        STX     .2,U            ; greeting (a label at the same place)
        RTS
```

which runs once, at the start. It's all automatic; it costs a few bytes of code for each such variable.

3.3 The direct page

```
__direct int frame;
__direct unsigned char flags;
```

puts the variables in the first 256 bytes of the data area, where DP points, and each use is a byte shorter and a cycle faster (`LDD <_frame` rather than `LDD _frame,U`). Use it for the variables a program uses most; all of them together must fit in 256 bytes.

4 The command line and the exit

```
#include <stdio.h>

int main(argc, argv)
int argc;
char **argv;
{
    int i;

    for (i = 0; i < argc; i++)
        printf("%d: %s\r", i, argv[i]);
    return argc - 1;
}
```

- **`argv[0]` is the module's name,** and the rest are the words of the parameters `F$Fork` passed, split at blanks; `"..."` keeps words together. `args one "two three"` gives `args`, `one` and `two three`.
- **The value `main` returns,** or that `exit()` is given, is the status `F$Exit` passes back: the error code the parent's `F$Wait` gets, as it would any command's. A program that succeeds returns 0.

5 Lines, the terminal and files

5.1 Lines end with CR

OS-9 ends a line with a carriage return, `'\r'`, and bcc09 doesn't translate it: write `"\r"` where a Unix program writes `"\n"`. On a terminal, SCF adds the line feed. Reading, a line ends with `'\r'` too:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char *line = 0;
    size_t size = 0;
    int n = 0;

    while (getdelim(&line, &size, '\r', stdin) > 0)
        n++;
    printf("%d LINES\r", n);
    return 0;
}
```

`getline()` and `fgets()` stop at `'\n'`, which OS-9's lines don't have; `getdelim()` with `'\r'` reads them a line at a time.

5.2 The paths

`stdin`, `stdout` and `stderr` are paths 0, 1 and 2, which the program gets from its parent, redirections and all. `read()` and `write()` are `I$ReadLn` and `I$WritLn`: on a terminal, SCF edits and echoes a line at a time, and `read()` gives at most one line, up to its CR. `isatty()` is true for an SCF device, so `stdout` is buffered a line at a time there, and in blocks into a file.

5.3 Files

`fopen()`, `open()`, `lseek()`, `unlink()` and `remove()` are RBF's calls: `open()` is `I$Open`, or `I$Create` with `O_CREAT` (and `O_EXCL`, `O_TRUNC`, `O_APPEND`); a new file can be read and written by its owner and read by others, whatever the mode says (`attr` changes that). A path is OS-9's: relative to the data directory (`chd`), or from a device, `/D0/USR/notes`.

```
#include <stdio.h>

int main(argc, argv)
int argc;
char **argv;
{
```

```
    FILE *fp;
    int c, lines = 0, chars = 0;

    if (argc != 2) {
        fprintf(stderr, "USAGE: COUNT FILE\r");
        return 1;
    }
    if ((fp = fopen(argv[1], "r")) == 0) {
        fprintf(stderr, "CAN'T OPEN %s\r", argv[1]);
        return 1;
    }
    while ((c = getc(fp)) != EOF) {
        chars++;
        if (c == '\r')
            lines++;
    }
    fclose(fp);
    printf("%d LINES, %d CHARACTERS\r", lines, chars);
    return 0;
}
```

OS-9's errors become `errno`'s `ENOENT`, `EBADF`, `EACCES`, `EEXIST`, `ENOMEM`, or, for the rest, `EIO`.

5.4 Directories

`mkdir()` is `I$MakDir`, `chdir()` is `I$ChgDir` for the data directory, and `opendir()`, `readdir()`, `rewinddir()` and `closedir()` read RBF's directory entries (`d_ino` is the file descriptor's LSN):

```
#include <stdio.h>
#include <dirent.h>

int main(argc, argv)
int argc;
char **argv;
{
    DIR *d;
    struct dirent *e;

    if ((d = opendir(argc > 1 ? argv[1] : ".")) == 0) {
        fprintf(stderr, "CAN'T READ THE DIRECTORY\r");
        return 1;
    }
    while ((e = readdir(d)) != 0)
        if (e->d_name[0] != '.')
            printf("%-29s %6ld\r", e->d_name, e->d_ino);
    closedir(d);
    return 0;
}
```

OS-9 has no calls for `rmdir()` and `getcwd()`, which do what the `deldir` and `pwd` commands do: `rmdir()` checks that the directory is empty, turns off its `D` attribute (`SS.Attr`, Level 2's and NitrOS-9's) and deletes it; `getcwd()` climbs `..` to the root and gives a path such as `/D1/USR/SRC`.

6 Memory

The data area holds the program's data, its BSS, and the stack, whose size `objcopy09 -d` gives (1K by default; a program with big local arrays, or deep recursion, needs more):

```
objcopy09 -O os9 -n prog -d 4096 prog.out prog
```

The heap that `malloc()` uses grows past the end of the data area with `F$Mem`, a page at a time. On Level 2 and NitrOS-9, each process has its own 64K; on Level 1 the data area can only grow if the memory after it is free, so a program that must run there shouldn't count on `malloc()`, and should keep what it needs in arrays. Memory `malloc()` gets goes back to OS-9 when the program exits.

7 System calls of your own

The C library makes the calls it needs (`I$Open`, `I$ReadLn`, `F$Mem` and the others); for any other, write a function in assembly. bcc09's assembler, `as09`, has an `OS9` instruction, `SWI2` followed by the call's code, and takes OS-9's names with a `$` in them:

```
; int os9time(char *buf): F$Time, the date and time into buf's
; (time() does this, as a time_t; this is an example)
; 6 bytes (year - 1900, month, day, hours, minutes, seconds);
; 0, or OS-9's error code
F$Time  equ     $15
        .text
        export  _os9time
_os9time:
        os9     F$Time          ; X: the buffer, the first argument
        bcs     err
        ldx     #0
        rts
err:    clra                    ; the error code, in B
        tfr     d,x
        rts

; unsigned os9sleep(unsigned ticks): F$Sleep, for ticks (0: until a
; signal); the ticks left
F$Sleep equ     $0A
        export  _os9sleep
_os9sleep:
        os9     F$Sleep         ; X: the ticks
        rts                     ; X: the ticks left
```

- **The first argument is in X,** the rest on the stack above the return address (`2,S`, `4,S` and so on); the result goes back in X.
- **U must be the same at the end:** it points at the data. A call that takes U saves it first (`PSHS U` … `PULS U`).
- **DP must be the same too**, and any of the others may change.
- **The code must be position-independent:** `LBSR` and `LEAX label,PC`, no absolute addresses of its own; its variables, if it has any, are `name,U`.

Assembled with the program (`bcc09 -Mo -O -o clock.out clock.c os9time.s`), the functions are called from C:

```
#include <stdio.h>

int os9time(char *buf);
unsigned os9sleep(unsigned ticks);

int main()
{
    unsigned char t[6];
    int i;

    for (i = 0; i < 3; i++) {
        if (os9time(t) != 0)
            return 1;
        printf("%02d:%02d:%02d\r", t[3], t[4], t[5]);
        os9sleep(60);       /* a second, at 60 ticks */
    }
    return 0;
}
```

8 What's there, and what isn't

- **There:** all of C89's library: `stdio` (with `printf`'s and `scanf`'s whole formats, floating point among them, and `tmpfile()`), `string.h`, `ctype.h`, `stdlib.h`, `math.h`, `setjmp.h`, `assert.h`, and `time.h`, whose `time()` reads OS-9's clock (`F$Time`; local time, taken as UTC); and `stdint.h`, `stdbool.h`, and the file and directory calls above.
- **There only as far as C needs:** signals, raised only by `raise()` and `abort()` (OS-9's own signals don't reach the program), and `getenv()`, which finds nothing.
- **Not there:** starting other processes (`F$Fork` and `F$Wait`), and `rename()`; write them in assembly, as above, if a program needs them.
- **Floating point** is in software, exact, and optional: a program that doesn't use it doesn't carry it, and one that prints a `double` carries about 6K more for it.

9 Testing

- **sim6809 –os9** runs a module on your PC, emulating the system calls the C library makes (`I$Open` and the rest, on the host's files, and `F$Mem`), so a program that only uses those can be tried in a moment:

```
sim6809 --os9 --param "one two" args
```

- **NitrOS-9 in an emulator,** with the module copied to a disk image, is the real thing: bcc09's own tests run their modules on NitrOS-9 Level 2 on an emulated CoCo 3, through a startup file that runs each one and writes its output to the disk.
- `os9 ident` checks a module's header and CRC.

Appendix: from OS-9 assembly to C

In assembly	In C
`leax msg,pcr` / `ldy #len` / `lda #1` / `os9 I$WritLn`	`write(1, msg, len);` or `printf(...)`
`fcc "TEXT"` / `fcb $0D`	`"TEXT\r"`
`os9 I$Open`	`open(name, O_RDONLY)` or `fopen(name, "r")`
`os9 I$Create`	`open()` with `O_CREAT`, or `fopen(name, "w")`
`os9 I$ReadLn`	`getdelim(&line, &size, '\r', fp)`
`os9 I$Delete`	`unlink(name)`
`os9 I$MakDir` / `I$ChgDir`	`mkdir(name, 0755)` / `chdir(name)`
`os9 F$Mem`	`malloc(n)`
`clrb` / `os9 F$Exit`	`return 0;` from `main`, or `exit(0)`
`rmb 2` in the data area	`int x;`
`ldd x,u`	`x`
`ldd <x`	`x`, declared `__direct`
the parameters at X	`argc` and `argv`